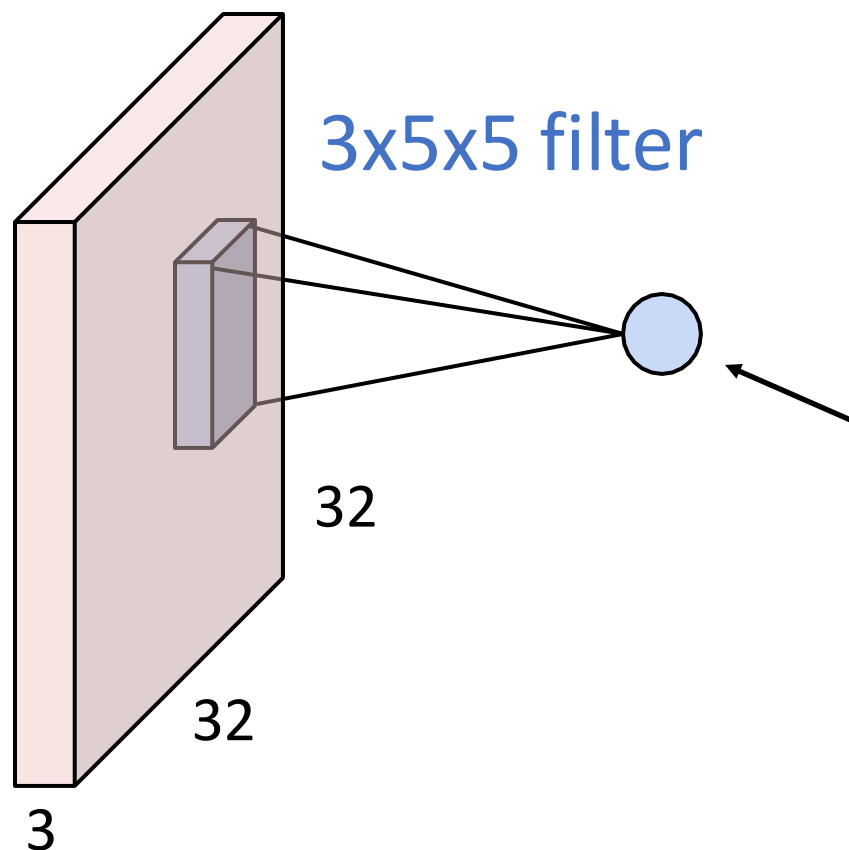


Convolution Layer

3x32x32 image

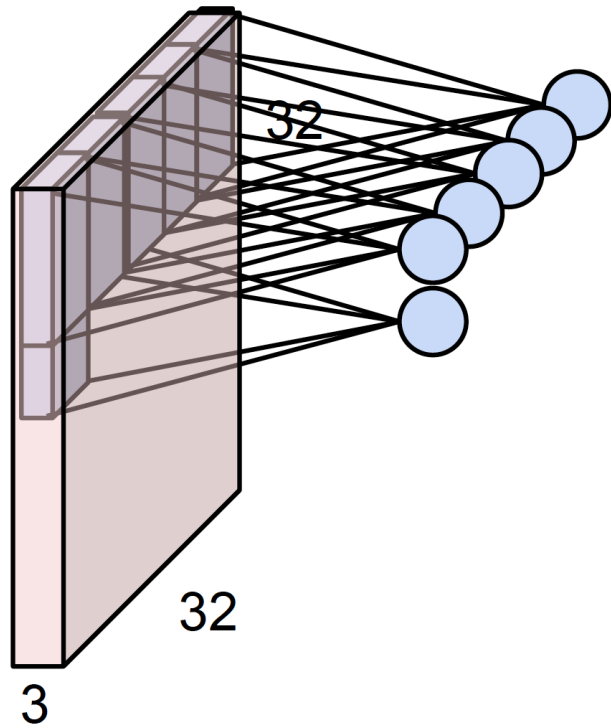


1 number:

the result of taking a dot product between the filter and a small 3x5x5 chunk of the image
(i.e. $3 \times 5 \times 5 = 75$ -dimensional dot product + bias)

$$w^T x + b$$

Convolutional Layer



Commonly: add padding

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Very common:
Set $P = (K - 1) / 2$ to
make output have
same size as input!

Strided Convolution

Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

Convolution Example

Input volume: **3** x 32 x 32

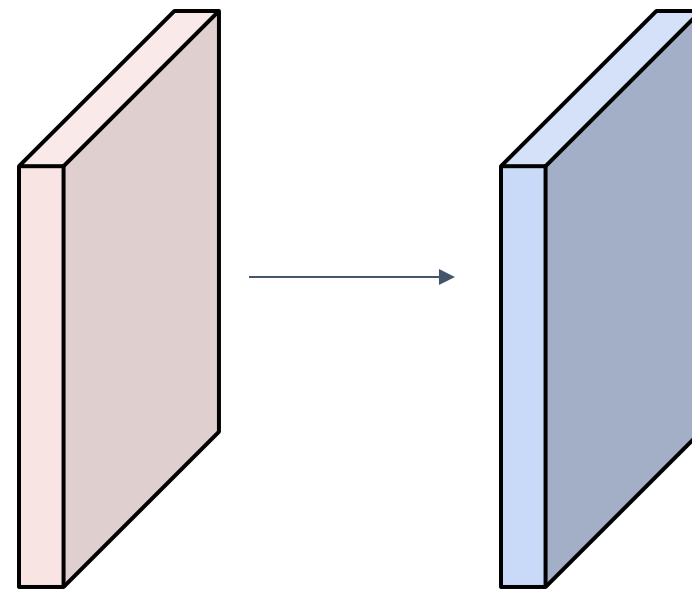
10 **5x5** filters with stride 1, pad 2

Output volume size: **10 x 32 x 32**

Number of learnable parameters: 760

Number of multiply-add operations: **768,000**

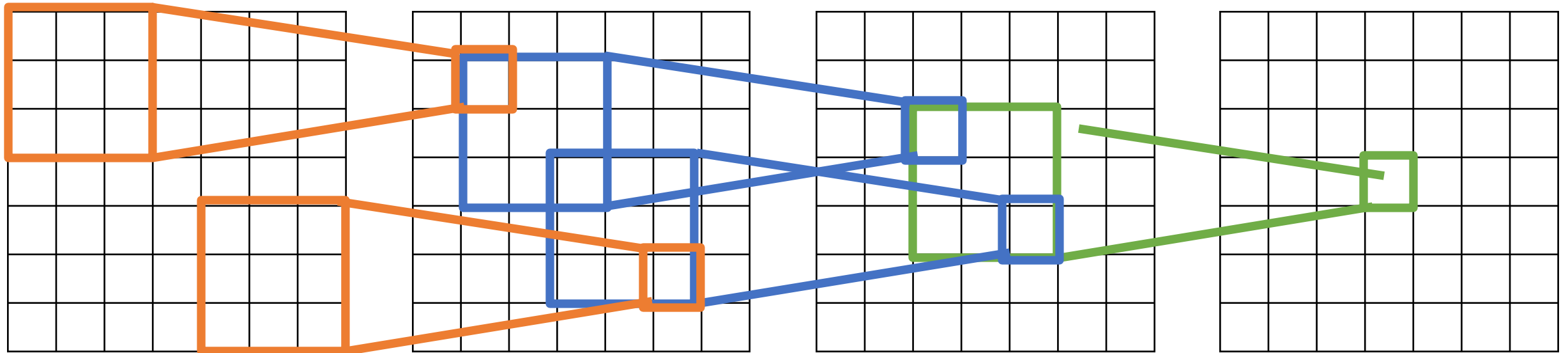
10*32*32 = 10,240 outputs; each output is the inner product of two **3x5x5** tensors (75 elems); total = $75 * 10240 = \mathbf{768K}$



Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size

With L layers the receptive field size is $1 + L * (K - 1)$



Input

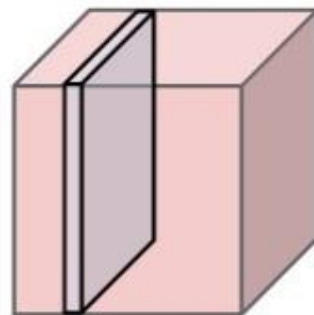
Problem: For large images we need many layers
for each output to “see” the whole image

Solution: Downsample inside the network

Output

Max Pooling

224x224x64



Single depth slice

x ↑

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

→ y

Max pooling with 2x2
kernel size and stride 2

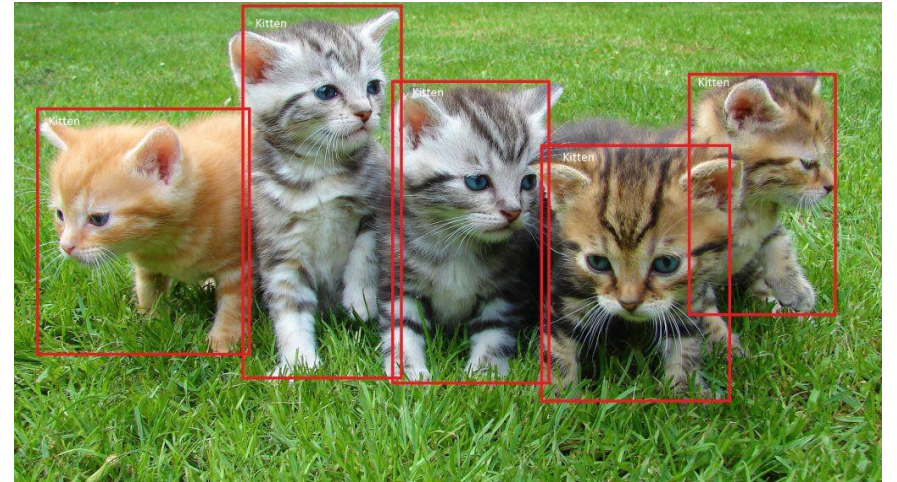


6	8
3	4

Introduces **invariance** to
small spatial shifts

Translational equivariance

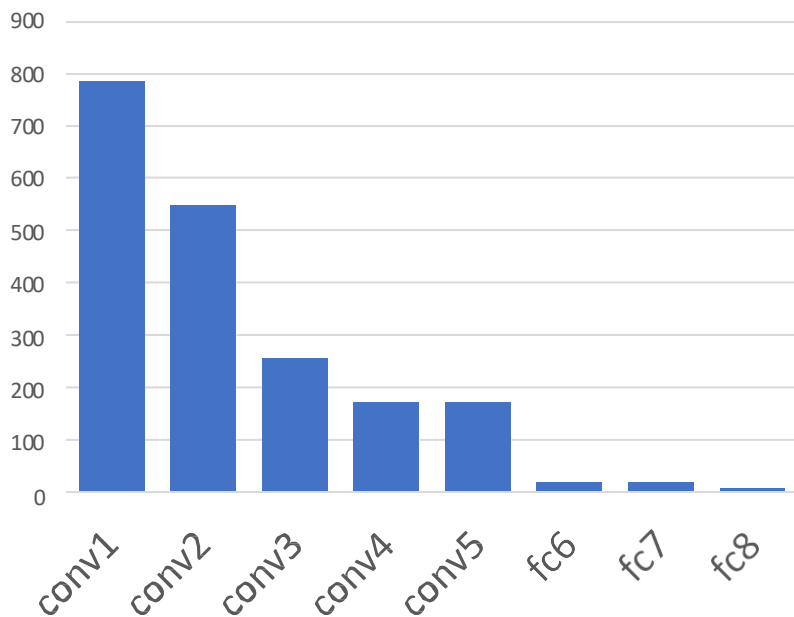
- The filter detecteds patterns irrespective of position in the image.



AlexNet

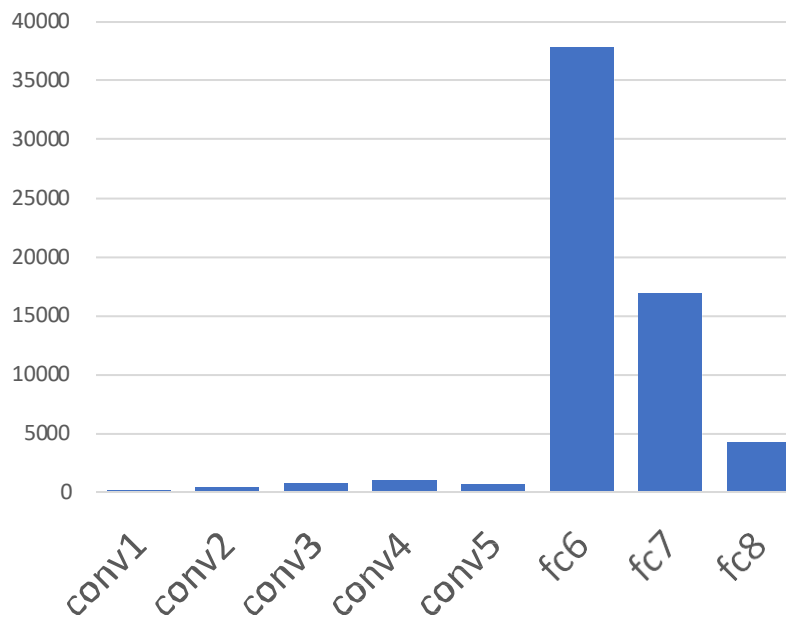
Most of the **memory usage** is in the early convolution layers

Memory (KB)



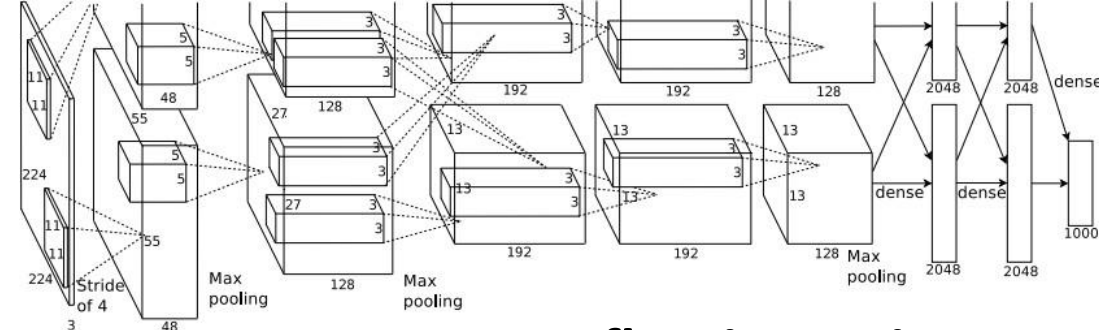
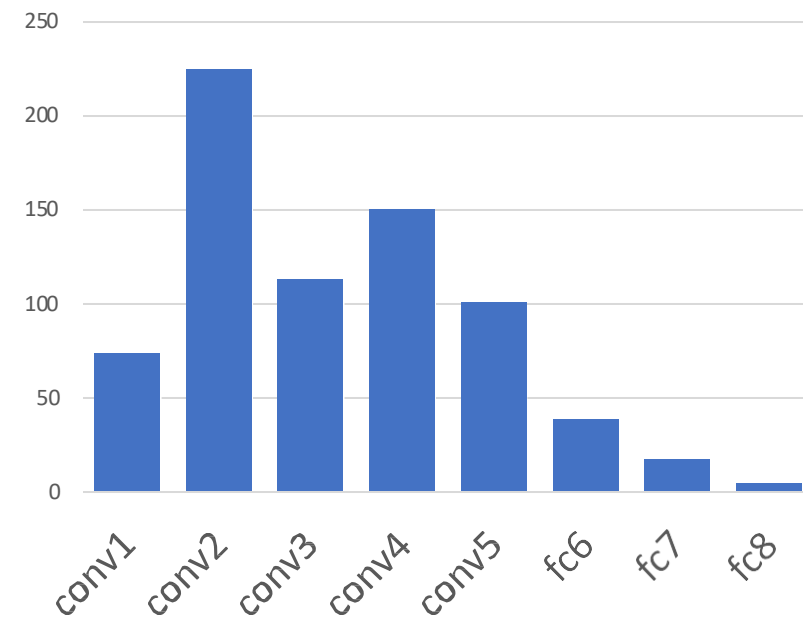
Nearly all **parameters** are in the fully-connected layers

Params (K)



Most **floating-point ops** occur in the convolution layers

MFLOP



VGG: Deeper Networks, Regular Design

VGG Design rules:

- All conv are 3x3 stride 1 pad 1
- All max pool are 2x2 stride 2
- After pool, double #channels

Input: $C \times 2H \times 2W$

Layer: Conv(3x3, $C \rightarrow C$)

Memory: $4HWC$

Params: $9C^2$

FLOPs: $36HWC^2$

Input: $2C \times H \times W$

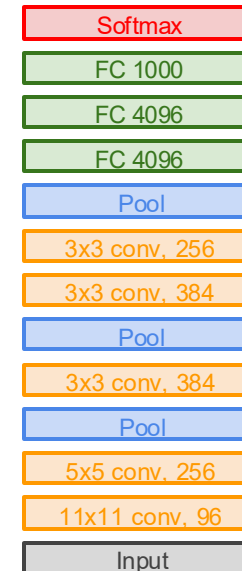
Layer: Conv(3x3, $2C \rightarrow 2C$)

Memory: $2HWC$

Params: $36C^2$

FLOPs: $36HWC^2$

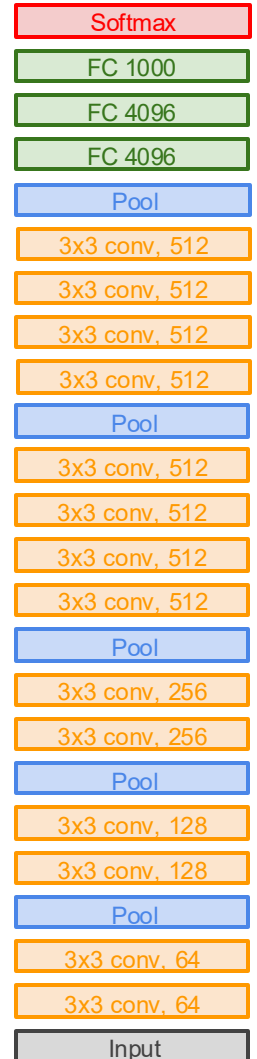
Conv layers at each spatial resolution take the same amount of computation!



AlexNet



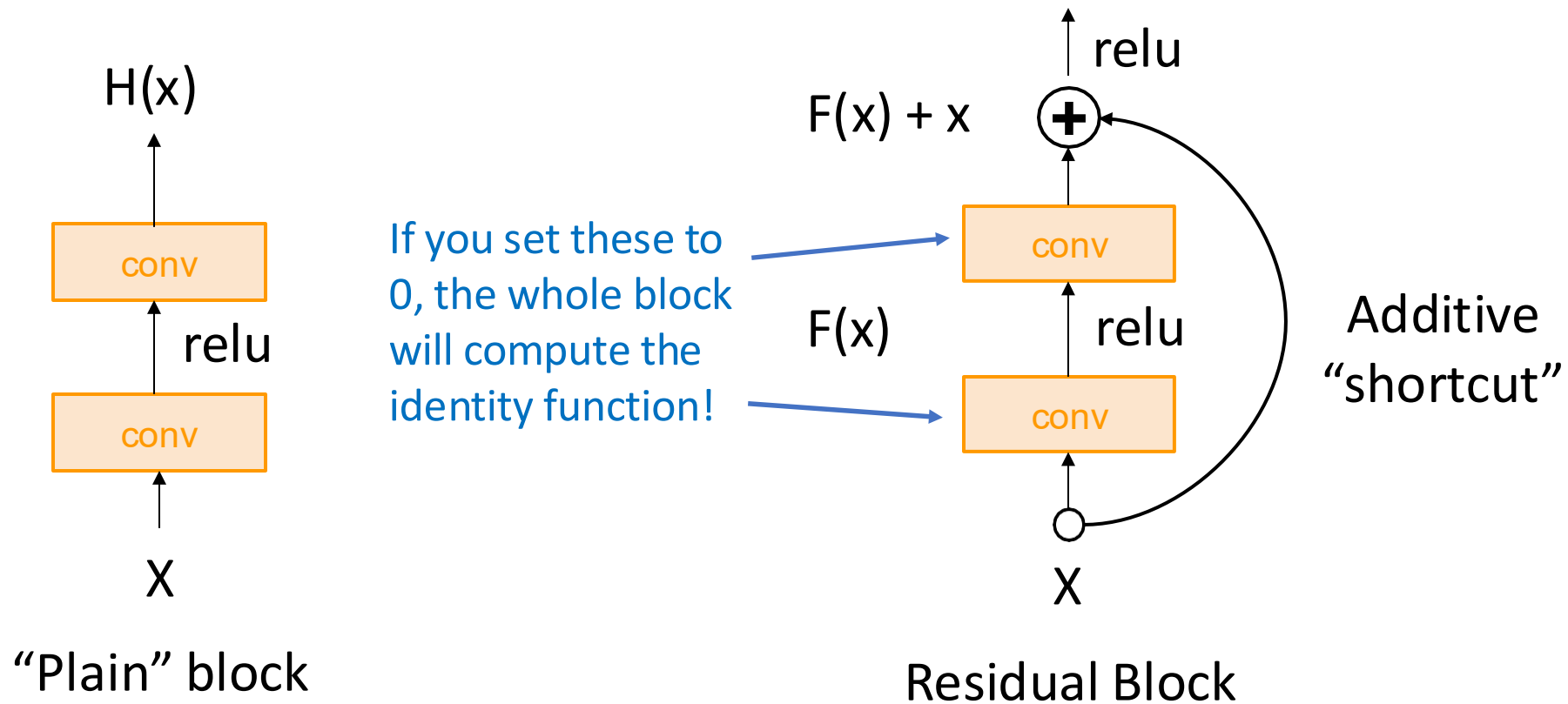
VGG16



VGG19

Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!



Residual Networks

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear



ImageNet top-5 error: 8.58

GFLOP: 3.6

ResNet-34:

Stem: 1 conv layer

Stage 1: 3 res. block = 6 conv

Stage 2: 4 res. block = 8 conv

Stage 3: 6 res. block = 12 conv

Stage 4: 3 res. block = 6 conv Linear

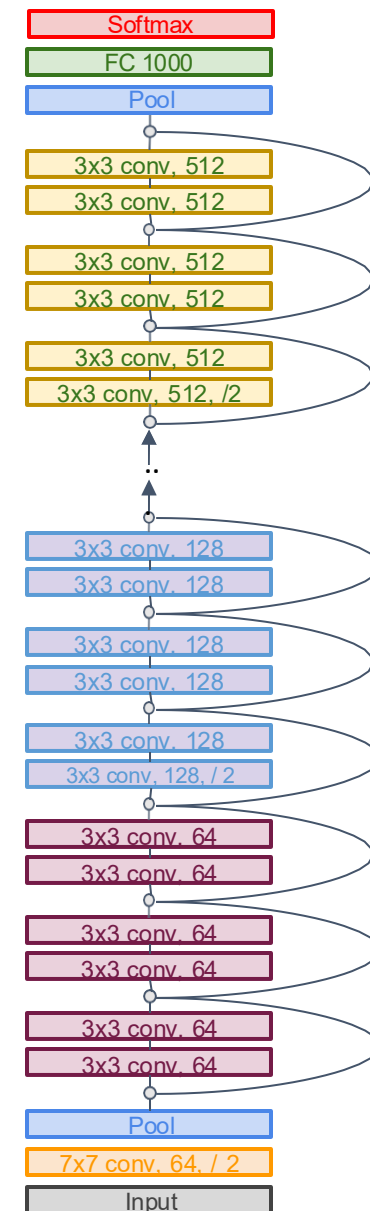


ImageNet top-5 error: 8.58

GFLOP: 3.6

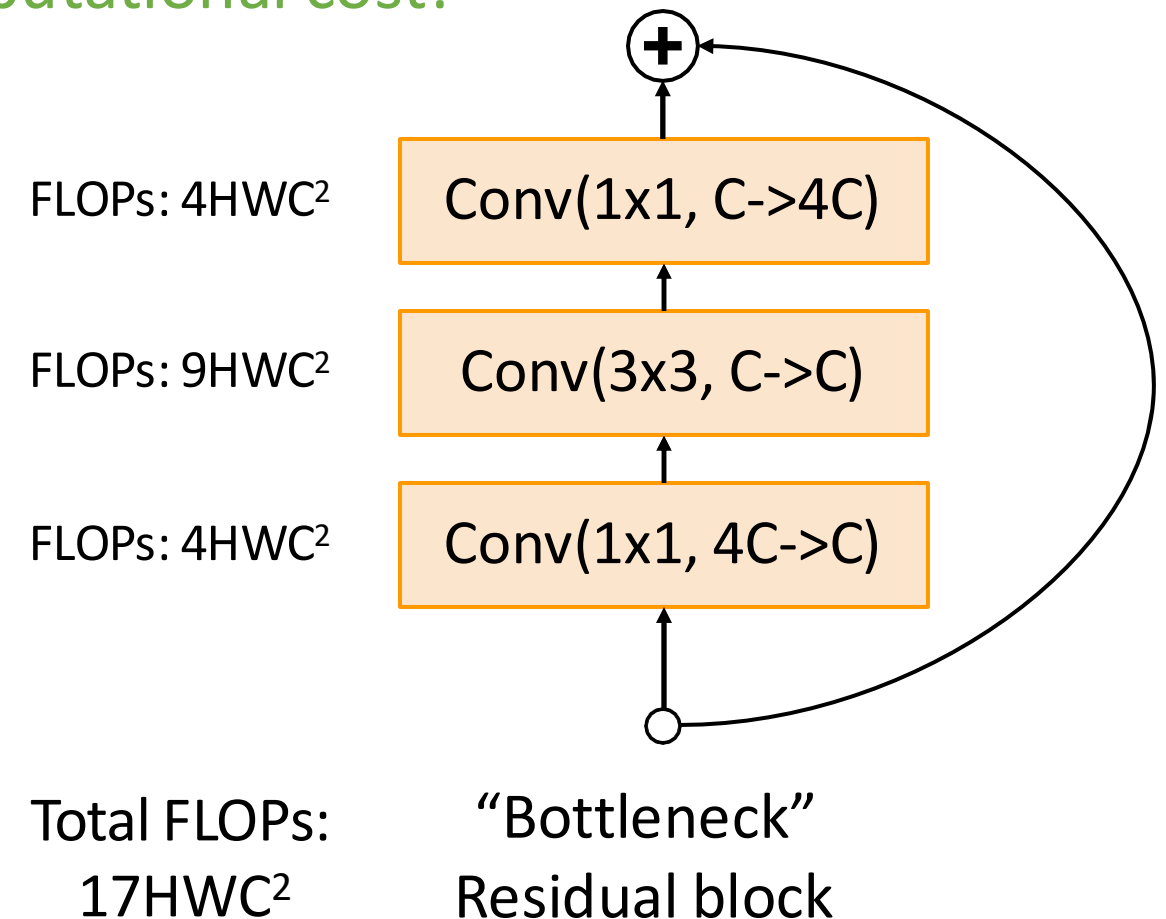
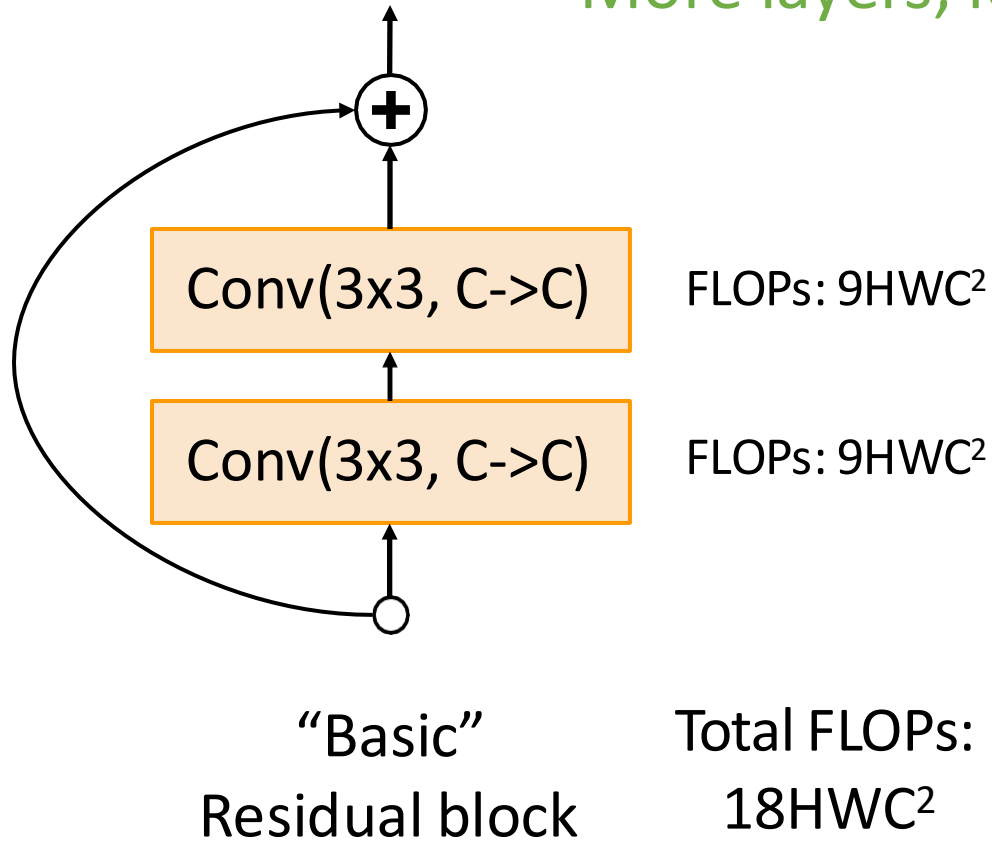
VGG-16:

ImageNet top-5 error: 9.62 GFLOP: 13.6

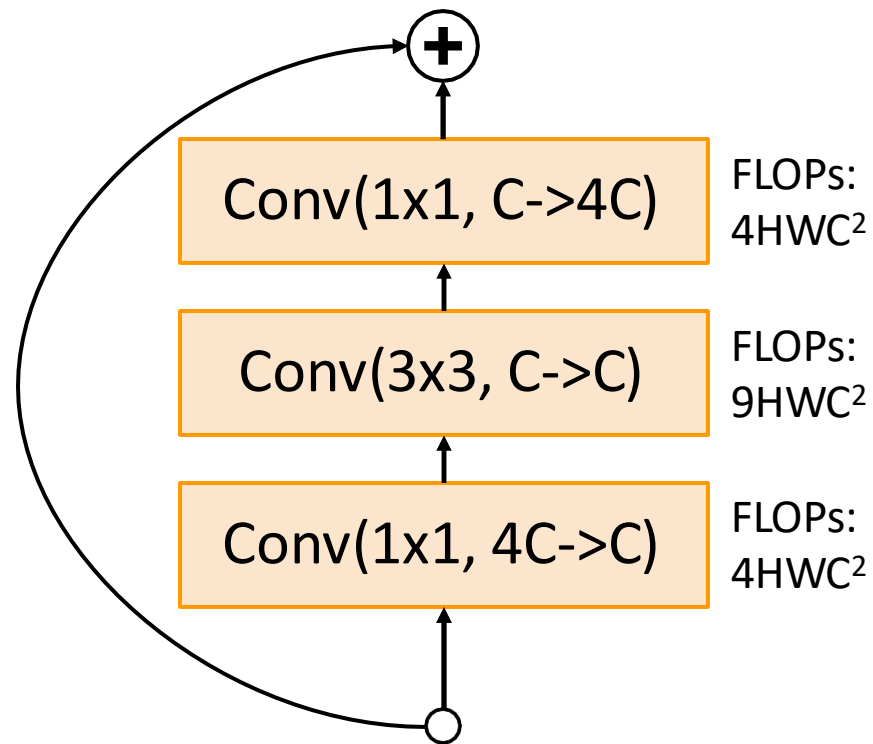


Residual Networks: Bottleneck Block

More layers, less computational cost!

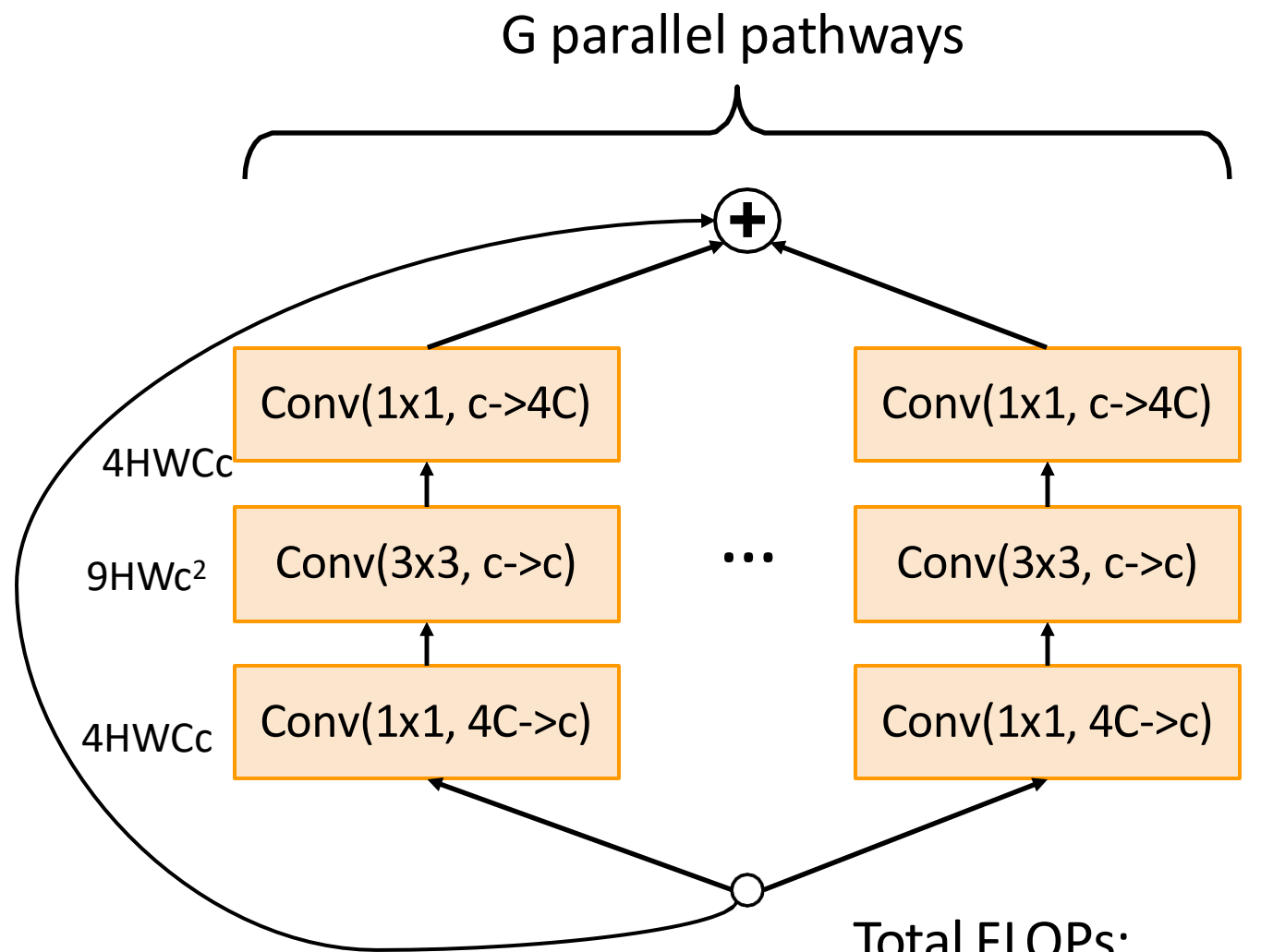


ResNeXt



“Bottleneck”
Residual block

Total FLOPs:
 $17HWC^2$



Equal cost when
 $9Gc^2 + 8GCc - 17C^2 = 0$

Example: $C=64, G=4, c=24$; $C=64, G=32, c=4$

Grouped Convolution

Convolution with groups=1:

Normal convolution

Input: $C_{in} \times H \times W$

Weight: $C_{out} \times C_{in} \times K \times K$

Output: $C_{out} \times H' \times W'$

FLOPs: $C_{out} C_{in} K^2 H W$

All convolutional kernels touch
all C_{in} channels of the input

Depthwise Convolution

Special case: $G=C_{in}$, $C_{out} = n C_{in}$

Each input channel is convolved
with n different $K \times K$ filters to
produce n output channels

Convolution with groups=G:

G parallel conv layers; each “sees”
 C_{in}/G input channels and produces
 C_{out}/G output channels

Input: $C_{in} \times H \times W$

Split to $G \times [(C_{in} / G) \times H \times W]$

Weight: $G \times (C_{out} / G) \times (C_{in} \times G) \times K \times K$

G parallel convolutions

Output: $G \times [(C_{out} / G) \times H' \times W']$

Concat to $C_{out} \times H' \times W'$

FLOPs: $C_{out} C_{in} K^2 H W / G$